Cytomachia User Manual

Getting Started:

To access Cytomachia, it is currently not hosted anywhere and must be ran locally.

Running Locally:

On a computer with NodeJS, Angular CLI, and a working GPU, clone the Cytomachia project from https://github.com/SeanButters/Cytomachia.

Open a terminal in the cloned repository. Then change directory to /cytomachia:

```
PS C:\Sean\code\Cytomachia> cd cytomachia
```

Run *npm i* to install all necessary packages:

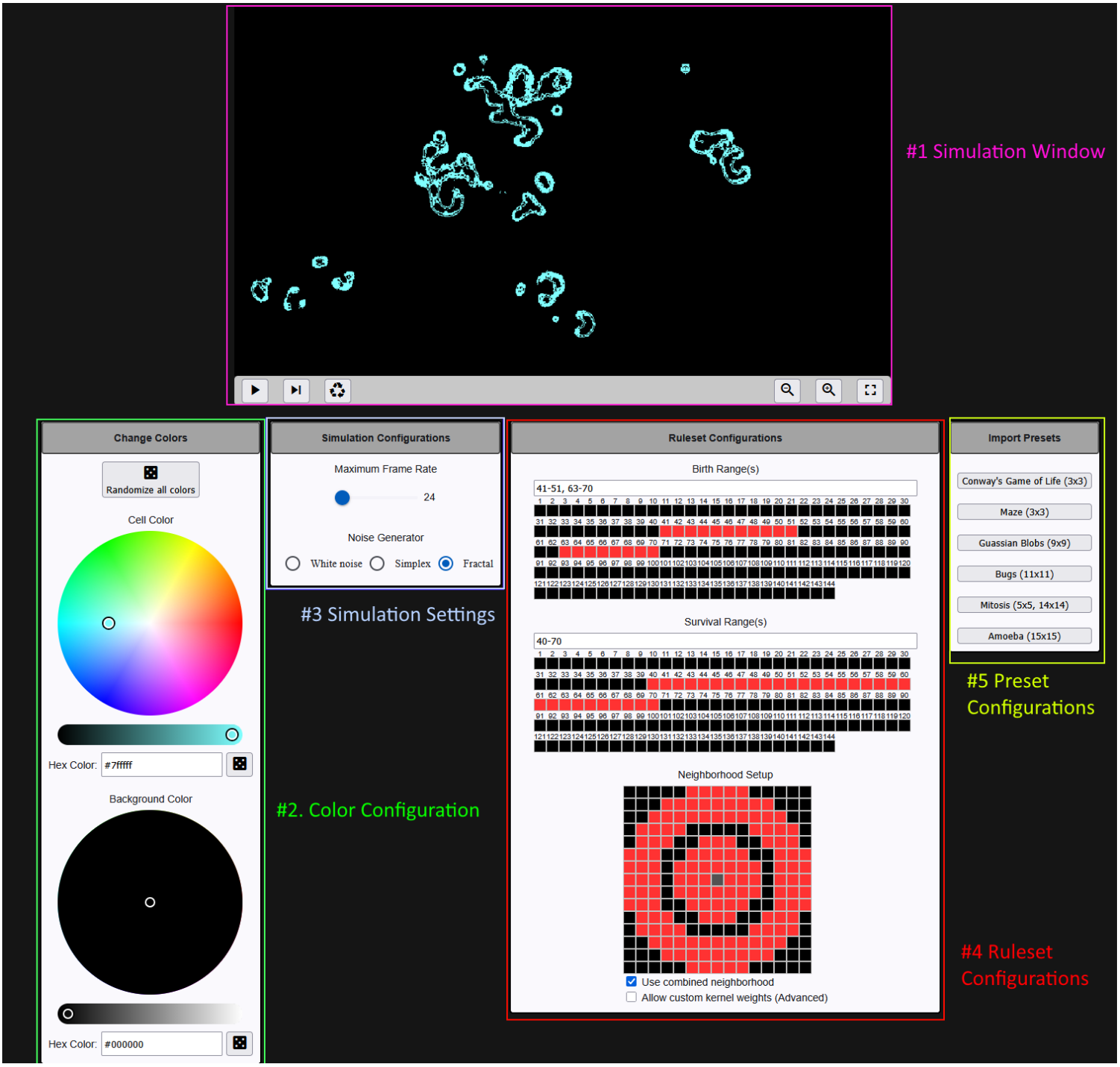
```
PS C:\Sean\code\Cytomachia\cytomachia> npm i

up to date, audited 542 packages in 10s
```

Then, run *ng serve* to host the application locally:

```
PS C:\Sean\code\Cytomachia\cytomachia> ng serve
Initial chunk files | Names         |  Raw size
main.js             | main          | 139.64 kB |
styles.css          | styles        |   8.95 kB |

                    | Initial total | 148.59 kB

Application bundle generation complete. [1.571 seconds] - 2026-04-09T01:12:09.477Z

Watch mode enabled. Watching for file changes...
NOTE: Raw file sizes do not reflect development server per-request transformations.
9:12:09 PM [vite] (client) Re-optimizing dependencies because lockfile has changed
  ➜  Local:   http://localhost:4200/
  ➜  press h + enter to show help
```

On your preferred browser, navigate to http://localhost:4200/ to start using the application

Once the application is open, when fully zoomed out, it should look roughly like this:



While the simulation inside the simulation window and rulset configuration may differ on your initial start up, the layout of the page should remain the same. The colored highlights around the section are not part of the application but helpful guides to break apart the different components of the application.

Application Basics:

The application is broken down into 2 main components, the simulation window (#1), and then the configurations (#2,#3,#4,#5). The simulation window is how you view and direcly interact with the simulation, while the configuration is settings you can edit that will affect *how* the simulation runs. Before going into detail about the various components and settings, this guide will discuss the gerneral setup and premise behind the simulation it self.

The Simulation:

The simulation is 2D cellular automata engine that runs on a 2560 x 1440 grid with around ~3.7 million cells, that will be updated every frame. The grid is toroidal, meaning the edges wrap (top to bottom) (side to side), like a donut; most visible when the simulation is fully zoomed out.

Each cell of the simulation has a state: Alive, or Dead. Each compute cycle, every cell within the simulation will have its state updated, defined by the rules configuration (#4) and the local surroundings of the cell.

In this application, the rules governing the cells behavior each cycle are:

Birth rule: If a cell is dead this cycle, how many alive neighbors does it need to be 'born' into the alive state in the next cycle

Surivial Rule: If a cell is alive this cycle, how many alive neighbors does it need to stay in the alive state the next sycle

Neighborhood: The area around the cell that is counted when calculating a cells neighbors. The maximum range is 7 cells in a square pattern from the current cell.

These states are constant across every cell within the application, and while simple once you understand the premise, the end result of each simulation, is almost garunteed to never be the same as a previous one with the same setup.

This simulation does support editing the cellular automata rules at run-time, allowing you to directly influence the cell population at run-time.

The simulation itself is run locally on your device's GPU, and does require a GPU to run properly.

The Simulation Window (#1):

This is the main tool used to view and interact with the simulations you create with this application. It is dominated by the viewport, Which renders the camera's view of the simulation behind the curtains.

Ineracting with the viewport

You can direcly interact with the view port to update the cameras position and zoom. With your mouse over the viewport, scrolling will zoom the camera in/out at your cursor's location, while clicking and dragging will move the camera around the simulation.

Simualtion Window buttons

At the bottom of the simulation window, there is a bar of buttons that can affect the state of the simulation or how the window is presented.
From the left, the leftmost button is the pause/play button, which will toggle the simulation compute loop from running to being paused.

The next button, will step the simulation one, allowing you to see how the automata evovles at youre own pace.

To the right of that is the Reseed button, denoted by its recycle symbol. When pressed, the simulation will be completely reseeded by the noice generator in the simulation configurations, and start its life over again.

One the other side of the button bar, you will find buttons that affect your view of the simulation. The furthest left of these, is the zoom out button, which will zoom the camera out at set intervals.

Similarly, to the right of that is the zoom in button, which will zoom the camera in at set intervals.

On the far right, is the expanded view toggle. By default the size of the simulation window is capped to allow for better access to the configurations. But want to view your automata in full glory? Click the button to enter expanded view mode, which will render the simulation window as large as it can on your screen. Combined will fullscreen mode with f11, it alows for a peaceful viewing experience.

Keyboard Controls

The appliation also supports keyboard controls for the simulation:

Arrow Keys: the arrow keys will move the camera around in the simulation world

Spacebar: The spacebar will toggle the pause option, allowing quick access to the feature

N: pressing n will quickly trigger the Step Once effect, similar to the button

R: Pressing r will trigger the Reseed operation to reinitialize the simulation

Configurations (#2,#3,#4,#5):

The applications configurations are broken into two categories. Configuations that affect the simulation (how it looks and how it runs), and configurations that affect the automata (the rules the cells follow). The configuraiton windows on the left (Color configuration and simulation configuration) will handle the simulation and application settings, while the two configuration windows on the right (ruleset configuration) handle the rulesets the automata cells follow during the simulation.

Color configuration (#2)

Wish to change the color of the simulation? Use the color wheels and sliders to adjust the colors of both foreground alive cells and the background. The simulation also accepts hex RGB codes for input, aswell as color ouput.

Feeling lucky? Use the dice buttons to update the foreground cells or the background or both to randomly genreated colors!

Simulation Configuration (#3)

These settings affect the max frame rate the simulation computation uses, as well as settings for affecting the noise generator used by the Reseeding algorithm.

Maximum frame cap: This number will represent the maximum FPS that the simulation compute algorithm runs at, if you want to run faster for smoother visuals, or slower if you are woried about GPU limitations. This setting is adjustable using the slider, in increments of 8.

Noise Generator: This setting is used by the simulation when running the Reseed function. It determines the algorithm used when generating the initial state for the simulation. White noise is truly random, providing an even spread of random inputs, simplex noise often produces geometric patterns within the automota, while fractal noise produces the most varied results, at the cost of extra computing time for generating its 4D input noise.

Ruleset configuration (#4)

This configuration window contains all of the settings used by the automata cells during the simulation. The simulation will react to any changes in real-time, allowing you complete control of how your simulaiton evolves and grows. It contains the 3 Rules that decide the state of the automata cells each iteration: The birth rule, the survival rule, and the neighborhood kernels used for counting neighbors

Birth Range(s)

This set of inputs will determine the birth rule the automata cells use. When a cell is dead, it will count its neighbors then look at this rule to determine whether that cell will be alive in the next iteration. The rule can be edited both by the editing the text input, or by clicking one of the indivual cells underneath the text input. The text input accepts a list of ranges, seperated by commas (ex. 4-6, 8, 12-16), from 1, up to the the maximun number of possible neighbors. Any updates to the text input should be reflected in the list of mask cells below and vice versa. The list of cells is number, and the color of the mask cells indicates if that number of neighbors is accepted by the automata rules; Red is active, black is inactive numbers. Clicking a cell will active/deactivate that specific number of neighbors for the birth rule.

Survival Range(s)

Similar to the birth ranges section, this set of inputs denotes the automata rule for the survival rule: If a cell is alive in the current iteration, how many neighboring alive cells does it need to stay alive into the next iteration. It similarly accepts a text input for a list of ranges, or editing by clicking the indiviual cells from the list of mask cells below.

Neighborhood Setup

This section allows you to edit what the automata cells view as neighbors, up to a range of 7 cells from the current cell, in a square pattern, for a total kernel size of 15x15. The current active neighborhood(s) are displayed as a grid, with active neighboring cells that will count to the total neighborhood marked in red, while inactive neighboring cells are marked in black. The center of the grid is marked grey and demarcates where the current active cell would be located when the neighbor count algorithm is ran. In the neighborhood setup's default mode, clicking on a individual cell of the neighborhood kernel will toggle the cell from the active/inactive states.

Underneath are two settings:

Use Combined Kernel:

On by default, this determines whether the simulation uses the same neighborhood to count neighbors when the cell is alive and dead. Toggling this setting off, will separate the neighborhood kernels used by the birth and survival rules, giving more flexibility to the final simulation. The buttons in the middle of the two kernels allow you to quickly copy one kernel to another or swap them. The two seperated kernels are able to be edited as normal by clicking the cells to toggle them on or off, for the neighborhood counting algorithm in the simulation.

Allow Custom Kernel Weights:

Off by default, this setting determines whether the kernel neighborhoods allow for custom weights. By default, all cells within the neighborhood counting algorithm have the same weight in the final neighborhood count of a cell, but with this option, you are able to indiviually edit kernel weights from values 0 up to a maximum weight of 9. Cell weights of zero are not counted in the final neighbor count for a cell, while a weight of 9 gives a cell 9 times the importance of a cell with a weight of 1 when counting neighbors for a particular cell. Clicking on an individual cell will allow you to edit the weight of that cell, and once you are editing one cell, using the arrow keys will let you quickly move around the neighborhood cell to quickly edit and submit values.

Preset Configurations

This section contains a list of buttons that will active a preset ruleset for the simulation, chosen as either notable cellular automata such as Conway's game of life, or 'Bugs' from the 'Larger then Life' set of cellular automata, but also includes a number of preset rulesets determined as interesting by the creator. Clicking one of these presets will automatically load the values for that preset in to the Ruleset Configuration UI, and update the simulation to match, even while running. These can provide a good launching off point for designing neighborhood kernels, as many presets contain larger and more complex kernels than the default simulation.

Tips for designing cellular automata rules

Trying to find a set of cellular automa rules that form an interesting simulation can be tricky, but if you want to create your own , I would recommend starting with designing your neibhorhood kernel(s) first, as that will determine the valid ranges for the birth and surival rules. When first testing out a neighborhood, generally safe options is to setup a birth range that covers the central part of the valid range, with the surival rule covering a similar but slightly larger range, where you can then edit from there.

If the simulation is looking too sparse, temporarily adding a low number to the birth rule such as 1, 2, or 3, depending on neighborhood size, will quickly increase the cell population within the simulation, where you can then remove that temporary value once you're happy with the densisty of the population. Similarly temporarily removing values from the birth range is the fastest way of reducing the cell population. While not recommended for long term automata stability, these options are generally the fastest way of growing or shrinking the desnity of the automata simulation.

Designing for split kernels

When designing split kernels, (separate kernels for birth and surival rules), its useful to know the general behaviors the kernel shapes of each manifest. The birth kernel is the main driver of motion within the simulation. Shfiting a given birth rule kernel to the right will produce leftward movement in the simulation, and vise versa with shifting the kernel to the left. Simmilarly shifting the kernel down will produce upward movement in the simulation while shifing the kernel up will produce downward movement in the simulation.

While it can be difficult to design rules that produce a stable population, you will most likely end up with a rule that will lose or gain populaiton over time. The difference in shape between the birth rule and survival rule kernels will dictate how the cellular automata grows or decays. Having a small birth kernel, but a wide survival kernel with a gap in the middle, will create groups of cells that will separate from each other. Feel free to try how other combinations work and influence simulation behavior.

Other notable tips

When first designing a rule, its generally best to edit while the simulation is paused, then do fine adjustments later while the simulation is running to judge how small chagnes affect the simulation behavior.

When loading some preset configurations, based on the previous rule you had, your simulation my completely empty. Don't worry, just use the Reseed feature to quickly fill the simulation up with valid cells.